

可修改性

面向对象

职责、协作

三个特性：封装、继承、多态

结构化

自顶向下

数据流图-结构图-流程图-代码

Outline

- 可修改性
- 继承vs组合
- 继承和构造方法

可修改性

动物园程序
每一种动物用一个类表示

Client代码

创建类的代码

```
Animal[] animals = new Animal[5];
```

```
animals [0] = new Dog();  
animals [1] = new Cat();  
animals [2] = new Wolf();  
animals [3] = new Hippo();  
animals [4] = new Lion();
```

使用类的代码

```
for (int i = 0; i < animals.length; i++) {  
    animals[i].eat();  
    animals[i].sleep();  
}
```

关于Client代码默认的知识

- 大量的
- 分散的
- 如果发生修改重新编译的话，是需要大量的时间的

需求的变更

- 狗eat的行为发生改变
- 新的一种动物加入进来
- 创建一个能放任何动物的list

可修改性

- (狭义) 可修改性
 - 对已有实现的修改
- 可扩展性
 - 对新的实现的扩展
- 灵活性
 - 对实现的动态配置

实现可修改性对Client代码的影响

- (狭义) 可修改性
 - 对已有实现的修改
 - 希望不影响Client代码
- 可扩展性
 - 对新的实现的扩展
 - 希望不影响Client使用类的代码
- 灵活性
 - 对实现的动态配置
 - 希望不影响Client使用类的代码

实现的修改——狗eat行为发生修改

- class Dog extends Animal{
- public void eat(){
- • • •
- • xxx
- }
- }

可扩展性—新的动物

- `animal[4] = new Panda();`

灵活性-//可以放指向任何继承Animal的动物

- `class Zoo{`
- `List<Animal> petlist;`
- `}`

继承vs组合

Inheritance

```
class Fruit {  
    //...  
}  
  
class Apple extends Fruit {  
    //...  
}
```

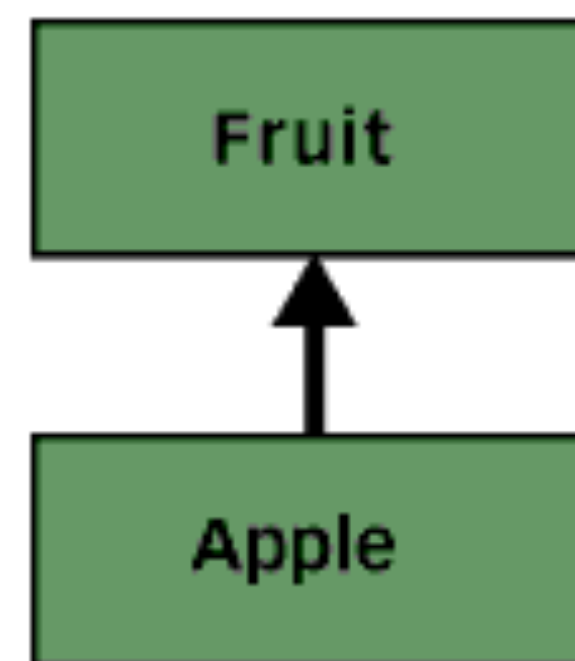


Figure 1. The inheritance relationship

Composition

```
class Fruit {  
    //...  
}  
  
class Apple {  
    private Fruit fruit = new Fruit();  
    //...  
}
```

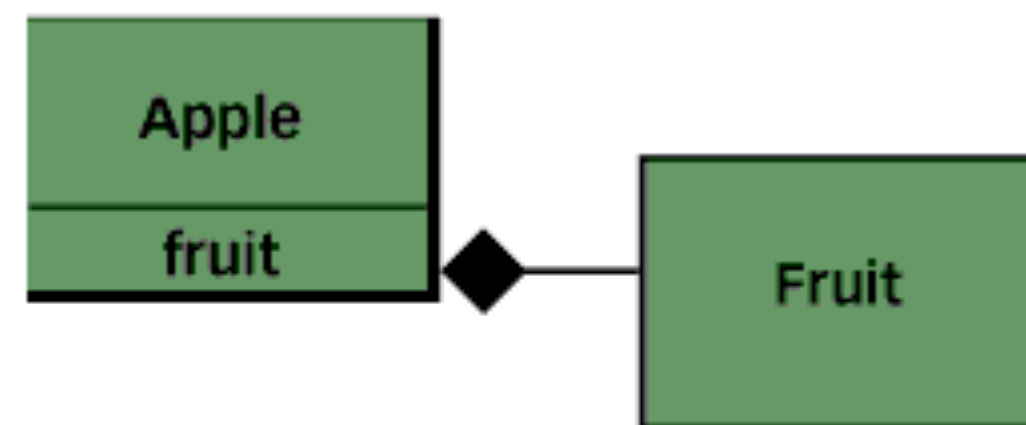


Figure 2. The composition relationship

继承和组合的选择

- 组合和继承都允许你在新的类中设置子对象（subobject），组合是显式地这样做的，而继承则是隐式的。

修改代码

- 增加新子类
 - 继承使代码更容易更改
- 修改父类
 - 对父类的一个小小的更改就会波及到应用程序代码中的许多其他地方，并需要对其进行更改。

Code reuse via inheritance I

```
class Fruit {  
    // Return int number of pieces of peel that  
    // resulted from the peeling activity.  
    public int peel{  
        System.out.println("Peeling is appealing.");  
        return 1;  
    }  
}  
  
class Apple extends Fruit {}  
  
class Example1 {  
    public static void main (String[] args) {  
        Apple apple = new Apple();  
        int pieces = apple.peel();  
    }  
}
```

Code reuse via inheritance II

```
class Peel {
    private int peelCount;

    public Peel (int peelCount) {
        this.peelCount = peelCount;
    }

    public int getPeelCount {
        return peelCount;
    }
    //...
}

class Fruit {
    // Return a Peel object that
    // results from the peeling activity.
    public Peel peel {
        System.out.println ("Peeling is appealing. ");
        return new Peel(1);
    }
}
```

// Apple still compiles and works fine
class Apple extends Fruit {

}

*// This old implementation of Example 1
// is broken and won't compile.*

```
class Example1 {
    public static void main (String[] args) {
        Apple apple = new Apple();
        int pieces = apple.peel();
    }
}
```

Code reuse via composition I

```
class Fruit {  
    // Return int number of pieces of peel that  
    // resulted from the peeling activity.  
    public int peel{  
        System.out.println("Peeling is appealing.");  
        return 1;  
    }  
}  
  
class Apple {  
    private Fruit fruit = new Fruit();  
  
    public int peel() {  
        return fruit.peel();  
    }  
}  
  
class Example2 {  
    public static void main (String[] args) {  
        Apple apple = new Apple();  
        int pieces = apple.peel();  
    }  
}
```

Code reuse via composition II

```
class Peel {  
    private int peelCount;  
  
    public Peel (int peelCount) {  
        this.peelCount = peelCount;  
    }  
  
    public int getPeelCount() {  
        return peelCount;  
    }  
    //...  
}  
  
class Fruit {  
    // Return int number of pieces of peel that  
    // resulted from the peeling activity.  
    public Peel peel() {  
  
        System.out.println("Peeling is appealing.");  
  
        return new Peel(1);  
    }  
}
```

*// Apple must be changed to accomodate
// the change to Fruit*

```
class Apple {  
    private Fruit fruit = new Fruit();  
    public int peel {  
        Peel peel = fruit.peel();  
        return peel.getPeelCount();  
    }  
}
```

*// This old implementation of Example2
// still works fine.*

```
class Example1 {}  
  
    public static void main (String[] args) {  
        Apple apple = new Apple();  
        int pieces = apple.peel();  
    }  
}
```

Comparing composition and inheritance

- 更改后端类(组合)的接口比更改超类(继承)的接口更容易。正如前面的示例所示，更改后端类的接口需要更改前端类的实现，但不一定更改前端接口。只要前端接口保持不变，仅依赖于前端接口的代码仍然可以工作。相比之下，对超类接口的更改不仅可以基于继承层次结构向下波及到子类，还可以波及到仅使用子类接口的代码。
- 更改前端类(组合)的接口比更改子类(继承)的接口更容易。在不确保子类的新接口与其超类型兼容的情况下，不能仅仅更改子类的接口。例如，不能向子类添加具有相同签名但返回类型不同的方法作为继承自超类的方法。但是，组合允许更改前端类的接口，而不会影响后端类。

如何选择继承 or 组合？

- 确保继承为is-a关系建模
- 不要仅仅为了获得代码复用而使用继承
- 不要仅仅为了获得多态性而使用继承

组合的选择

- Private组合：将成员对象设为 private，外部无法直接访问。这是一种“黑盒”设计，实现了高度的信息隐藏。优势在于组件变更不影响外部，能显著提升系统的稳定性和安全性。
- Public 组合：成员对象设为 public，允许外部直接操作内部组件。这种方式接口透明，常用于需要暴露内部结构的场景（如 Car 类暴露 Engine），能清晰表达“部分-整体”的物理关系。

```
class 手机 {  
    // 把芯片藏在里面，用户看不到  
    private 芯片 xinPian;  
  
    public 手机() {  
        xinPian = new 芯片();  
    }  
  
    public void 开机() {  
        xinPian.启动(); // 用芯片的功能  
    }  
}
```

```
class 引擎 {  
    public void 启动() { ... }  
}  
class 车轮 {  
    public void 转动() { ... }  
}  
class Car {  
    // 公开零件，用户可以直接用  
    public 引擎 yinQing;  
    public 车轮 cheLun;  
  
    public Car() {  
        yinQing = new 引擎();  
        cheLun = new 车轮();  
    }  
}
```

继承和构造方法

如果子类继承了父类，
那么父类里面的变量应该什么时候初始化？

构造方法在继承体系中的执行过程

```
class Animal{  
    Animal(){  
        System.out.println("Animal");  
    }  
}
```

```
class Dog extends Animal{  
    Dog(){  
        System.out.println("Dog");  
    }  
}
```

```
new Dog();
```

Animal
Dog

如果先执行子类构造方法？

```
class Animal{  
    int age;  
  
    Animal(){  
        age = 10;  
    }  
}
```

```
class Dog extends Animal{  
    Dog(){  
        System.out.println(age);  
    }  
}
```

```

$ javap -verbose -c -private HelloWorld
Compiled from "HelloWorld.java"
public class HelloWorld extends java.lang.Object
    SourceFile: "HelloWorld.java"
    minor version: 0
    major version: 50
    Constant pool:
const #1 = Method #6.#15; // java/lang/Object."<init>":()V
const #2 = Field #16.#17; //
java/lang/System.out:Ljava/io/PrintStream;
const #3 = String #18; // Hello, world!
const #4 = Method #19.#20; //
java/io/PrintStream.println:(Ljava/lang/String;)V
const #5 = class #21; // HelloWorld
const #6 = class #22; // java/lang/Object
const #7 = Asciz <init>;
const #8 = Asciz ()V;
const #9 = Asciz Code;
const #10 = Asciz LineNumberTable;
const #11 = Asciz main;
const #12 = Asciz ([Ljava/lang/String;)V;
const #13 = Asciz SourceFile;
const #14 = Asciz HelloWorld.java;
const #15 = NameAndType #7:#8;// "<init>":()V
const #16 = class #23; // java/lang/System
const #17 = NameAndType #24:#25;//
out:Ljava/io/PrintStream;
const #18 = Asciz Hello, world!;
const #19 = class #26; // java/io/PrintStream
const #20 = NameAndType #27:#28;//
println:(Ljava/lang/String;)V
const #21 = Asciz HelloWorld;
const #22 = Asciz java/lang/Object;
const #23 = Asciz java/lang/System;

```

```

const #24 = Asciz out;
const #25 = Asciz Ljava/io/PrintStream;;
const #26 = Asciz java/io/PrintStream;
const #27 = Asciz println;
const #28 = Asciz (Ljava/lang/String;)V;

{
public HelloWorld();
    Code:
        Stack=1, Locals=1, Args_size=1
        0: aload_0
        1: invokespecial #1; //Method
java/lang/Object."<init>":()V
        4: return

    LineNumberTable:
        line 1: 0

public static void main(java.lang.String[]);
    Code:
        Stack=2, Locals=1, Args_size=1
        0: getstatic #2; //Field
java/lang/System.out:Ljava/io/PrintStream;
        3: ldc #3; //String Hello, world!
        5: invokevirtual #4; //Method
java/io/PrintStream.println:(Ljava/lang/String;)V
        8: return
    LineNumberTable:
        line 5: 0
        line 6: 8
}

```

如何调用父类构造方法？

```
class Dog extends Animal{  
    Dog(){  
        super();  
    }  
}
```

super()调用父类构造方法

```
class Dog extends Animal{  
    Dog(){  
        System.out.println("Dog");  
    }  
}  
  
Dog(){  
    super();  
    System.out.println("Dog");  
}
```

编译器自动添加super()

带参数父类构造器

```
class Animal{  
    Animal(String name){  
        this.name = name;  
    }  
}  
  
class Dog extends Animal{  
    Dog(){  
        super("旺财");  
    }  
}
```

执行顺序

Animal(String)
Dog()

this() 调用重载构造器

```
class Dog{  
    Dog(){  
        this("旺财");  
    }  
  
    Dog(String name){  
    }  
}
```

```
new Dog();
```

```
Dog(){  
    this("旺财");  
    super();  
}  
// 编译错误
```

类的初始化

- 一个类的初始化包括3个步骤：
 - 加载 (Loading)
 - 链接 (Linking)
 - 初始化 (Initialization)

加载

- (1) 加载类的二进制字节流文件(.class文件), 如果该类有基类, 向上一级一直加载到根基类(不管基类是否使用都会加载)。
- (2) 将二进制字节码加载到内存, 解析成方法区对应的数据结构。
- (3) 在java逻辑堆中生成该类的java.lang.Class对象, 作为方法区中该类的入口。

链接

- 验证：字节码完整性、final类与方法、方法签名等的检查验证。
- 准备：为静态变量分配存储空间(内存单元全置0，即基本类型为默认值，引用类型为null)。
- 解析(这步是可选的)：将常量池内的符号引用替换为直接引用。

类的加载和链接只执行一次，故static成员也只加载一次，作为类所拥有、类的所有实例共享。

类的初始化：

- 初始化静态字段(执行定义处的赋值表达式) `static int count = 10;`

- 执行静态初始化块

```
static{  
    System.out.println("初始化");  
}
```

注：有父类则先递归地初始化父类。

对象的初始化:

如果需要创建对象, 则会执行创建对象并初始化

(1) 在堆上为创建的对象分配足够的存储空间, 并将存储单元清零, 即基本类型为默认值, 引用类型为null。

(2) 初始化非静态成员变量(即执行变量定义处的赋值表达式)。

(3) 执行构造方法。

注: 如果有父类, 则先递归的初始化父类成员, 最后才是本类

Initialization with inheritance

1. 访问main(), 加载基类, 加载到根基类
2. 在根基类中进行静态初始化, 然后是下一个派生类, 依此类推
3. 所有成员变量赋默认值 (基本类型和引用设置为0或0.0 false null)
4. 调用基类构造函数
5. 成员变量按文本顺序初始化
6. 构造函数体的其余部分被执行

Initialization with inheritance

- 父类static变量
- 父类static代码块
- 子类static变量
- 子类static代码块
- 父类成员变量默认值
- 父类显式初始化
- 父类构造器
- 子类成员变量默认值
- 子类显式初始化
- 子类构造器

```
class Characteristic {  
    private String s;  
  
    Characteristic(String s) {  
        this.s = s;  
        System.out.println("Creating  
Characteristic " + s);  
    }  
  
    protected void dispose() {  
        System.out.println("finalizing  
Characteristic " + s);  
    }  
}
```

```
class Description {  
    private String s;  
  
    Description(String s) {  
        this.s = s;  
        System.out.println("Creating  
Description " + s);  
    }  
  
    protected void dispose() {  
        System.out.println("finalizing  
Description " + s);  
    }  
}
```

```
class LivingCreature {  
    private Characteristic p = new Characteristic("is alive");  
    private Description t = new Description("Basic Living Creature");  
  
    LivingCreature() {  
        System.out.println("LivingCreature()");  
    }  
  
    protected void dispose() {  
        System.out.println("LivingCreature dispose");  
        t.dispose();  
        p.dispose();  
    }  
}
```

```
class Animal extends LivingCreature {  
    private Characteristic p= new Characteristic("has heart");  
    private Description t = new Description("Animal not Vegetable");  
  
    Animal() {  
        System.out.println("Animal()");  
    }  
  
    protected void dispose() {  
        System.out.println("Animal dispose");  
        t.dispose();  
        p.dispose();  
        super.dispose();  
    }  
}
```

```
class Amphibian extends Animal {  
    private Characteristic p = new Characteristic("can live in water");  
    private Description t = new Description("Both water and land");  
  
    Amphibian() {  
        System.out.println("Amphibian()");  
    }  
  
    protected void dispose() {  
        System.out.println("Amphibian dispose");  
        t.dispose();  
        p.dispose();  
        super.dispose();  
    }  
}
```

```
public class Frog extends Amphibian {  
    private static Test monitor = new Test();  
    private Characteristic p = new Characteristic("Croaks");  
    private Description t = new Description("Eats Bugs");  
    public Frog() {  
        System.out.println("Frog()");  
    }  
    protected void dispose() {  
        System.out.println("Frog dispose");  
        t.dispose();  
        p.dispose();  
        super.dispose();  
    }  
    public static void main(String[] args) {  
        Frog frog = new Frog();  
        System.out.println("Bye!");  
        frog.dispose();  
    }  
}
```

```
monitor.expect(new String[] {  
    "Creating Characteristic is alive",  
    "Creating Description Basic Living Creature",  
    "LivingCreature()",  
    "Creating Characteristic has heart",  
    "Creating Description Animal not Vegetable",  
    "Animal()",  
    "Creating Characteristic can live in water",  
    "Creating Description Both water and land",  
    "Amphibian()",  
    "Creating Characteristic Croaks",  
    "Creating Description Eats Bugs",  
    "Frog()",  
    "Bye!",  
    "Frog dispose",  
    "finalizing Description Eats Bugs",  
    "finalizing Characteristic Croaks",  
    "Amphibian dispose",  
    "finalizing Description Both water and land",  
    "finalizing Characteristic can live in water",  
    "Animal dispose",  
    "finalizing Description Animal not Vegetable",  
    "finalizing Characteristic has heart",  
    "LivingCreature dispose",
```

```
    "finalizing Description Basic Living Creature",  
    "finalizing Characteristic is alive"  
});
```

```
abstract class Glyph {  
    abstract void draw();  
  
    Glyph() {  
        System.out.println("Glyph() before draw()");  
        draw();  
        System.out.println("Glyph() after draw()");  
    }  
}
```

```
class RoundGlyph extends Glyph {  
    private int radius = 1;
```

new RoundGlyph(5)?

```
    RoundGlyph(int r) {  
        radius = r;  
        System.out.println("RoundGlyph.RoundGlyph(), radius = " + radius);  
    }
```

```
    void draw() {  
        System.out.println("RoundGlyph.draw(), radius = " + radius);  
    }  
}
```

```
abstract class Glyph {  
    abstract void draw();
```

```
    Glyph() {  
        System.out.println("Glyph() before draw()");  
        draw();  
        System.out.println("Glyph() after draw()");  
    }  
}
```

```
class RoundGlyph extends Glyph {  
    private int radius = 1;
```

```
    RoundGlyph(int r) {  
        radius = r;  
        System.out.println("RoundGlyph.RoundGlyph(), radius = " + radius);  
    }
```

```
    void draw() {  
        System.out.println("RoundGlyph.draw(), radius = " + radius);  
    }  
}
```

```
"Glyph() before draw()",  
"RoundGlyph.draw(), radius = 0",  
"Glyph() after draw()",  
"RoundGlyph.RoundGlyph(), radius = 5"
```

构造方法中的多态

- 如果在构造函数中调用正在构造的对象的动态绑定方法会发生什么？
- 被覆盖的方法将在对象完全构造之前被调用

```
class Base {  
    private String baseName = "base";  
  
    public Base() {  
        callName();  
    }  
  
    public void callName() {  
        System.out.println(baseName);  
    }  
}
```

```
class Sub extends Base {  
    private String baseName = "sub";  
  
    public void callName() {  
        System.out.println (baseName) ;  
    }  
}
```

```
public static void main(String[] args) { Base b = new Sub(); }  
}
```

null

可能造成类被初始化的操作

- 创建一个Java类的实例对象
- 调用一个Java类的静态方法
- 为类或接口中的静态域赋值
- 访问类或接口中声明的静态域，并且该域的值不是常值变量
- 在一个顶层Java类中执行assert语句
- 调用Class类和反射API中进行反射操作

Example - 类的初始化

- class A{
- static int value = 100;
- static{
- System.out.println("类A初始化");
- }
- }
- class B extends A{
- static{
- System.out.println("类B初始化");
- }
- }
- }
- }
- public class StaticFieldInit{
- public static void main(String[] args){
- System.out.println(B.value);
- }
- }

- 结果：
- 类A初始化
- 100
- //当访问一个Java类或接口的静态域时，只有真正声明这个域类或接口才会被初始化

- class StaticBlock {
- static final int c = 3;
- static final int d;
- static int e = 5;
- static {
- d = 5;
- e = 10;
- System.out.println("Initializing");
- }

- StaticBlock() {
- System.out.println("Building");
- }
- }
- public class StaticBlockTest {
- public static void main(String[] args) {
- System.out.println(StaticBlock.c);
- System.out.println(StaticBlock.d);
- System.out.println(StaticBlock.e);
- }
- }

- 结果:
- 3
- Initializing
- 5
- 10

- 原因是这样的：
- 输出c时，由于c是编译时常量，不会引起类初始化，因此直接输出，输出d时，d不是编译时常量，所以会引起初始化操作，即static块的执行，于是d被赋值为5，e被赋值为10，然后输出Initializing，之后输出d为5，e为10。

常量在编译阶段会存入调用它的类的常量池中，本质上没有直接引用到定义该常量的类，因此不会触发定义常量的类的初始化：

```
class Const{  
    public static final String NAME = "我是常量";  
    static{  
        System.out.println("初始化Const类");  
    }  
}
```

```
public class FinalTest{  
    public static void main(String[] args){  
        System.out.println(Const.NAME);  
    }  
}
```

- 执行后输出的结果如下：
- 我是常量
- 虽然程序中引用了 `const` 类的常量 `NAME`，但是在编译阶段将此常量的值“我是常量”存储到了调用它的类 `FinalTest` 的常量池中，对常量 `Const.NAME` 的引用实际上转化为了 `FinalTest` 类对自身常量池的引用。也就是说，实际上 `FinalTest` 的 `Class` 文件之中并没有 `Const` 类的符号引用入口，这两个类在编译成 `Class` 文件后就不存在任何联系了。

通过数组定义来引用类，不会触发类的初始化：

- class Const{
 - static{
 - System.out.println("初始化Const类");
 - }
- }

- public class ArrayTest{
 - public static void main(String[] args){
 - Const[] con = new Const[5];
 - }
- }

- 执行后不输出任何信息，说明 Const 类并没有被初始化。
- 但这段代码里触发了另一个名为 “LLConst” 的类的初始化，它是一个由虚拟机自动生成的、直接继承于java.lang.Object 的子类，创建动作由字节码指令 newarray 触发，很明显，这是一个对数组引用类型的初始化，而该数组中的元素仅仅包含一个对 Const 类的引用，并没有对其进行初始化。

- class Const{
- static{
- System.out.println("初始化Const类");
- }
- }

- public class ArrayTest{
- public static void main(String[] args){
- Const[] con = new Const[5];
- for(Const a:con)
- a = new Const();
- }
- }

- 这样便会得到如下输出结果：
- 初始化Const类
- 根据四条规则的第一条，这里的 new 触发了 Const 类。

- class StaticBlock {
 - static {
 - d = 5;
 - e = 10;
 - System.out.println("Initializing");
 - }

StaticBlock.e
e值的变化?

- static final int d;

- static int e = 5;

- StaticBlock() {
 - System.out.println("Building");
 - }
- }

- 在这段代码中，对e的声明被放到static块后面，于是，e会先被初始化为10，再被初始化为5，所以这段代码中e会输出为5。